

Text Representation

1: John likes to watch movies. Mary likes movies too.

2: John also likes to watch football games.

Bag of words - המשפט הוא כמו סל עם אוסף של מילים - אין משמעות לסדר בין המילים.

Doc	John	likes	to	watch	movies	Mary	too	also	football	games
1	1	2	1	1	2	1	1	0	0	0
2	1	1	1	1	0	0	0	1	1	1

(2) **Big-grams** - **Bag of N-grams** - קיבוץ מילים לשמירת הקונטקסט נשתמש לדוגמה ב

Doc	John	likes	to	watch	movies	Mary	likes	movies	too	John	also	likes	watch	football	games
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0	0	1	1	1	1	1	1

Unigram - מילה 1 בכל עמודה.

TDM sparsity and Term sparsity

מטריצות שיש בהן הרבה אפסים. בגלל שיש מעט מילים - **sparsity** - כמה מרווח.

	1	2	3	4	5
R	0	0	1	1	0
Text	0	0	1	0	0
Stats	1	1	0	0	0
Mining	0	1	1	0	0
Book	0	0	0	0	1

$TDM\ sparsity = 17/25 = 0.68$

The term "Mining" sparsity = $3/5 = 0.6$

The term "Book" sparsity = $4/5 = 0.8$

$$Sparsity(TDM) = \frac{\text{Number of zeros}}{\text{Number of cells}}$$

$$Sparsity(Term) = \frac{\text{Number of zeros for the term}}{\text{Number of documents}}$$

Stem vs. Root vs. Lemma

Stem - קיצוץ התחלה או סוף של המילה.

Complete words - אחרי stemming פונקציה שמחזירה את ה stem למילה חוקית בשפה

Root - שינוי של המילה

Lemma - מילה חוקית במילון. זמן הרצה ארוך אבל הכי מדויק.

Stem	Root	Lemma
stem + affixes = valid word	doesn't have to be a word	valid word
Words: produce, production	Word: interrupt	Words: fly, flies, flew, flying
Stem = produc	Root = rupt	Lemma = fly

stemmer השיטה הטובה ביותר.

שלבים בניתוח כריית טקסט:

- הגדרת הבעיה והמטרות.
- הגדרת הטקסט הרלוונטי לניתוח.
- הטקסט "מלוכלך" וצריך לדעת נקודת ולהכין אותו.
- מארגנים את הטקסט בצורה שנוכל לנתח אותו.
- הפיצורים בשלב זה הם המשתנים הבלתי תלויים.
- הערכה כמה הפיצורים טובים מסקנות, תובנות, המלצות וכו'.

Text functions

- Nchar** - כמות תווים
- Subset** - הסרת
- Paste** - מחברת תווים
- Substr** - חילוץ מחזות קצרה
- Substring** - מחליפה טקסט.
- Grep** - חיפוש מילה בטקסט.
- Gsub** - מציאה והחלפה
- Strsplit** - הפיכת וקטור לרשימה

Document Term Matrix - **Corpus** = אוסף מסמכים
text <- c("This is a small example", "This is another example", "example appears many times", "small example")

1. צמצום המטריצה - מוריד את ה sparsity של מה שנמוך מ 0.7

smallDtm <- removeSparseTerms(DTM, .7)

2. **Tokenize** - הפרדה למילים ומשפטים. תיצור עמודות ותפריד מסמכים לפי מה שיש ב **BigramTokenizer**.

<<DocumentTermMatrix (documents: 4, terms: 7)>>
Non-/sparse entries: 12/16
Sparsity : 57%
Maximal term length: 7
weighting : term frequency (tf)

Term Document Matrix (Bag of Words & TF-IDF)

כל מילה בכל אחד מהמסמכים מופיעה באחת השורות, עבור כל מסמך יש עמודה וסופרים ושמם במטריצה כמה פעמים כל מילה מופיעה. שיוצרים TDM - יש פרמטר שנקרא waiting שמאופיין במשקלים.

Bag of Words - בודק כמה פעמים מילה מופיעה בכל מסמך - קובע משקל המילה **Weight = Term frequency**

ככל שהמילה מופיעה יותר היא פחות חשובה.

אם **Weight = Binary** בודקים רק אם המילה מופיעה.

ההבדל הבסיסי בין TDM ל DTM הוא שהשקלול של TDM מבוסס על תדירות המונחים (TF) ו DTM השקלול מבוסס על תדירות המסמך (TF-IDF).

ההבדל הבסיסי בין TDM ל DTM הוא שהשקלול של TDM מבוסס על תדירות המונחים (TF) ו DTM השקלול מבוסס על תדירות המסמך (TF-IDF).

Text preprocessing

		Before	After
tolower	Makes all text lowercase	Starbuck's is from Seattle.	starbuck's is from seattle.
removePunctuation	Removes punctuation like periods and exclamation marks	Watch out! That coffee is going to spill!	Watch out That coffee is going to spill
stripWhitespace	Removes tabs, extra spaces	I like coffee.	I like coffee.
removeNumbers	Removes numerals	I drank 4 cups of coffee 2 days ago.	I drank cups of coffee days ago.
removeWords	Removes specific words (e.g. he, and, were & she) defined by the data scientists	The coffee house and barista he visited were nice, she said hello.	The coffee house barista visited nice, said hello.
stemDocument	Reduces prefixes and suffixes on words making term aggregation easier.	Transforming words to do text mining applications is often needed.	Transform word to do text mine applic is often needed.

סדר שאינו נכון לניקוי: removePunctuation לפני הסרת אימוגים (שנותן טקסט לפני אימוגים) או removeWords לפני tolower.

TF-IDF -

הבלטת מילים שחוזרות על עצמן הרבה במסמך.

TF - כמה פעמים מילה מופיעה.

IDF - אם המילה נדירה זה מוסיף. המילה kindle

מופיעה 5 פעמים במסמך 1 מתוך 200 מילים במסמך.

$TF = 5/200 = 0.025$

קיימים 100 מסמכים. מתוכם 10 עם המילה kindle

$IDF = \log_2(100/10) = 3.32$

$TF \cdot IDF = 0.025 \cdot 3.32 = 0.083$

סוכמים ב tf-idf על מנת לבנות מערכת המלצה. נראה

כל הכתבות יהיו בצורת וקטורים (מסמך). נרצה למצוא

מסמכים שמכילים מילים דומות.

Subjectivity Lexicon

מילון דיפולטי שנותן ציון למילה - חיובי או שלילי.

Type	Length	Word	Part of Speech	Stemmed y/n	Polarity
Weak	1	Abundant	Adj	N	Positive
Weak	1	Abundance	Noun	N	Positive
Strong	1	Accede	Verb	Y	Positive
Weak	1	Accept	Verb	Y	Positive

Valence shifters

I do like herring <-> I do not like herring

1. **negator** - not משנה את המשמעות. אם נתקל ב not הוא יהפוך את הניקוד למינוס 1.

I like mango <-> I really like mango

2. **amplifier** - really מחזקת את הלייק ונרצה לחזק את הציון ולתת במקום 1 לדוגמה 1.5.

I like to drive <-> I hardly like to drive

3. **de-amplifier** - hardly המילה - לא שלילית אבל עדיין מורידה ציון (מחליש את המילה).

קיים מילון שיש לו מילה ועמודת קוד מספרי שאומר האם זה negator/ amplifier

Qdap's polarity function:

פונקציית sentiment תמיד מפרידה למשפטים ונותנת ציון לכל משפט בנפרד. אם משתמשים ב sentiment_by הציון יינתן לכל הקטע.

Amplifier "very" = +0.8 + Positive Polarity "good" = +1.0 = 1.8

Positive word cluster

S1 = "Sentiment analysis in R is very good."

7 words in total

$$Polarity(S1) = \frac{1.8}{\sqrt{7}} = 0.68$$

```
> sentiment('This apartment can be cleaner')
element_id sentence_id word_count sentiment
1: 1 1 5 0.2236068
```

```
> sentiment('This apartment can be cleaner',polarity_dt=updated_hash_senti
element_id sentence_id word_count sentiment
1: 1 1 5 -0.4472136
```

